The Latent Cognition Trilogy

Paulo Leocadio [1]

[1]Zinnia Labs, Broward County, FL, United States

Contents

Series Preface

The increasing scale and capability of contemporary neural architectures have exposed a persistent gap in their theoretical characterization. While substantial progress has been made in optimizing performance, scaling training regimes, and extending model capacity, the internal organization of learned representations remains only partially formalized. In particular, the interaction between memory, abstraction, and computation is often treated as a set of separate problems: memory as external retrieval, symbolic reasoning as emergent but unstructured behavior, and architectural routing as an engineering optimization. This fragmentation limits the ability to reason about model behavior within a unified conceptual frame.
The present work is motivated by the need for a more structured account of internal model dynamics. Rather than introducing a single empirical benchmark or an isolated architectural modification, the Trilogy adopts a conceptual and architectural approach to identify recurring patterns across modern neural systems. Its purpose is not to claim a complete theory of machine cognition, but to establish a minimal set of constructs through which memory, symbolic stabilization, and computational organization can be described as related dimensions of latent model behavior.
The Latent Cognition Trilogy is organized as a sequence of three investigations, each addressing a distinct but interdependent layer of internal computation. The first volume examines latent memory and the conditions under which retrieval may be understood as an intrinsic process rather than a purely external augmentation procedure. The second volume considers the emergence of symbolic stability under compression, curriculum structure, and representational constraint. The third volume addresses architectural organization, with particular attention to hierarchical routing mechanisms that coordinate computation across specialized components while preserving coherence at scale.
Taken together, these studies do not assume a unified theory in advance. They develop a layered description of internal model structure, moving

from memory to symbolic representation to architectural coordination. Each volume isolates a specific class of mechanisms and proposes corresponding formalizations, interpretive constructs, and testable propositions. The progression is therefore not intended as a claim of completeness, but as a structured pathway for analyzing latent cognition in artificial systems. This preface situates the individual volumes within that broader program. The sections that follow introduce the research-program note, the canonical relationship among the companion works, and the conceptual foundations shared across the Trilogy.

Note on the Research Program

The present manuscript belongs to a structured theoretical research program aimed at formalizing internal mechanisms of machine cognition. The program, titled *The Latent Cognition Trilogy*, is organized around three major stages: latent memory and retrieval-native representation, symbolic stabilization under compression and curriculum constraints, and architectural coordination through hierarchical routing mechanisms.

The Trilogy should not be read as an isolated sequence of essays. It is part of a broader research architecture that includes a foundational systems paper, companion hypothesis papers, and future domain-specific extensions. Within this architecture, *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function* functions as the architectural foundation. That work develops the systems-level premise that retrieval may be modeled as a native differentiable function within transformer-based systems rather than as an external procedural layer.

The three volumes of the Trilogy extend that architectural foundation into a broader theoretical sequence. Volume I examines latent memory and retrieval-native cognition. Volume II addresses the stabilization of symbolic structure within compressed latent spaces. Volume III analyzes hierarchical routing and architectural coordination as mechanisms for maintaining coherence across specialized computational components.

The companion hypothesis papers occupy a different position. They are not appendices to the Trilogy, nor substitutes for its main sequence. They function as satellite formalizations: each isolates a specific theoretical claim that supports, constrains, or extends the program without overloading the central manuscript.

Canonical Structure of the Research Program

The Latent Cognition Trilogy should be read as the main theoretical sequence within a broader research program on latent cognition, retrieval-native intelligence, and bounded artificial agency. The Trilogy is not an isolated manuscript collection, nor is it a container for every related hypothesis, architectural proposal, or companion paper. Rather, it functions as the central conceptual axis around which a wider body of work is organized.

Within this structure, *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function* serves as the architectural foundation of the research program. That work develops the systems-level premise that retrieval should not remain an external augmentation layer, as in conventional retrieval-augmented generation architectures, but may instead be modeled as a differentiable cognitive function embedded within the representational dynamics of transformer-based systems. For this reason, the DLI-LLM paper should be understood as the architectural substrate beneath the Trilogy rather than as a supplementary appendix.

The three companion hypothesis papers occupy a different position. They do not define the base architecture of the program. Instead, they formalize adjacent theoretical claims that support, constrain, or extend the main conceptual sequence developed in the Trilogy. Their purpose is to preserve analytical precision around claims that would otherwise overload the central manuscript.

Accordingly, the present work adopts the following canonical structure: the DLI-LLM paper functions as the foundational architecture paper; the Latent Cognition Trilogy forms the main theoretical sequence; and the companion hypothesis papers operate as formal satellite works within the same research program. This organization allows the Trilogy to remain

coherent while preserving the broader intellectual architecture required for future papers, extensions, and domain-specific applications.

Table 1: Canonical organization of the Latent Cognition research program.

Work	**Canonical Role**	**Relationship to the Trilogy**
Differentiable Latent Indexing in LLMs	Architectural foundation / formal paper version	Provides the systems-level substrate for Volume I and the broader Trilogy.
Volume I: Memory Fields	Main sequence, first volume	Develops retrieval-native cognition and latent memory fields.
Volume II: Symbolic Stabilization	Main sequence, second volume	Examines how compressed latent representations acquire symbolic stability.
Volume III: Hierarchical Routing	Main sequence, third volume	Analyzes routing as architectural coordination across specialized computation.
Companion Hypothesis Paper I	Formal companion work	Isolates and formalizes a supporting theoretical claim.
Companion Hypothesis Paper II	Formal companion work	Extends the framework into an adjacent conceptual domain.
Companion Hypothesis Paper III	Formal companion work	Tests or constrains the boundaries of the main theoretical sequence.
Future extensions	Parallel works	Apply or expand the framework into cybersecurity, conscious AI, health sciences, bounded autonomy, or governance-aware AI systems.

Conceptual Foundations

Core Constructs

The Trilogy operates on a set of shared constructs that describe internal representations and computational organization in neural architectures. These constructs are not intended to form a closed ontology. They provide a working vocabulary for examining how memory, symbolic stabilization, and routing may interact within latent computational systems.

Latent State. A latent state is represented as $\mathbf{h}_t \in \mathcal{L}$, encoding the internal condition of the model at computational step t. It is not treated merely as an intermediate activation, but as a structured representational state through which information, context, and transformation history may be partially expressed.

Memory Field. A memory field, denoted $\mathcal{M}$, refers to a structured collection of latent representations accessible through differentiable retrieval. Within the Trilogy, the memory field provides the conceptual basis for treating retrieval as an internal representational operation rather than as an external lookup procedure.

Symbolic Representation. A symbolic representation is denoted as $s \in \mathcal{S}$ and refers to a compressed or stabilized encoding derived from latent states. The term does not imply classical symbolic manipulation in the strict sense. Rather, it designates a representational condition in which compressed latent structures acquire sufficient stability to support abstraction, reuse, and interpretation.

Routing Function. A routing function, denoted $\mathcal{G}(\cdot \mid \mathcal{E})$, governs the allocation of computation across specialized modules, experts, or representational pathways. In this framework, routing is treated not only as an efficiency mechanism, but as a structural principle through which large-scale architectures maintain coherence across distributed computation.

Structural Relationship

The Trilogy examines the interaction between these constructs across three stages:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

This progression should not be interpreted as a strict causal chain. Rather, it represents the analytical sequence adopted by the Trilogy. The first stage examines how latent memory may be structured and retrieved. The second stage considers how compressed representations may acquire symbolic stability. The third stage analyzes how routing mechanisms coordinate computation across specialized structures.

The shared hypothesis across the Trilogy is that these constructs are not independent engineering conveniences. They may represent interdependent layers of a broader latent cognitive architecture: memory supplies retrievable structure, symbolic stabilization supplies abstraction, and routing supplies coordinated computational organization.

Chapter 1

Overview of the Trilogy

1.1 Purpose of the Trilogy

The purpose of *The Latent Cognition Trilogy* is to provide a structured theoretical account of latent computation in contemporary neural architectures. The Trilogy does not attempt to define consciousness, agency, or intelligence in general terms. Its focus is narrower and more architectural: it examines how memory, symbolic stabilization, and routing may operate as interdependent layers within large-scale neural systems.

The central motivation is that many current descriptions of artificial intelligence remain divided between external engineering mechanisms and internal cognitive interpretation. Retrieval is often treated as an external service. Symbolic behavior is often described as an emergent property without sufficient structural explanation. Routing is often framed as an optimization mechanism for scaling model capacity. The Trilogy proposes that these components can be examined together as part of a latent computational architecture.

1.2 Volume I: Latent Memory and Retrieval-Native Cognition

Volume I examines memory as an intrinsic dimension of latent computation. It develops the idea that retrieval may be modeled not only as an external augmentation process, but as an internal operation over latent representational fields. This volume is informed by the architectural foundation developed in *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*, but it extends that foundation

into a broader theoretical discussion of memory, access, and latent state organization.

1.3 Volume II: Symbolic Stabilization under Compression

Volume II addresses the conditions under which latent representations acquire symbolic stability. Rather than assuming that symbolic structure appears automatically as an emergent property of scale, this volume examines the role of compression, curriculum, constraint, and representational reuse in shaping stable abstractions. Its central concern is the transition from distributed latent representation to reusable symbolic structure.

1.4 Volume III: Hierarchical Routing and Architectural Coordination

Volume III examines routing as a mechanism of architectural coordination. In large-scale systems, computation is increasingly distributed across specialized components, expert modules, external tools, memory systems, and policy layers. This volume analyzes how hierarchical routing may preserve coherence across such distributed structures and how routing mechanisms may function as part of a broader latent cognitive architecture.

1.5 Relation to Companion Works

The Trilogy is supported by companion works that clarify, formalize, or extend its claims. The DLI-LLM paper provides the architectural foundation for retrieval-native cognition. The companion hypothesis papers isolate specific theoretical claims that orbit the Trilogy without being absorbed into it. This separation allows the main sequence to remain coherent while preserving the broader research program required for future development.

1.6 Scope and Limits

The Trilogy is not presented as a completed theory of artificial cognition. It is a structured research pathway. Its aim is to define a coherent vocabulary, organize related mechanisms, and propose a set of formal and

conceptual relationships that can guide future empirical, architectural, and philosophical work.

Volume I: Memory Fields

Differentiable Latent Indexing inside Large Language Models

Volume I Abstract

This article proposes *Differentiable Latent Indexing* (DLI), a theoretical framework that recontextualizes memory retrieval not as an external utility appended to a language model, but as a native, differentiable function inside the computational pathway of Large Language Models (LLMs). By treating the latent space as a structured information field, the article derives formal retrieval operators that integrate memory access directly into the model's forward pass.

The central claim is that the conventional separation between storage and processing introduces architectural inefficiencies that limit the development of autonomous and cognitively organized AI systems. In current retrieval-augmented designs, the model, retriever, embedding store, and generator often remain operationally coupled but mathematically discontinuous. DLI instead proposes a shared latent memory substrate in which retrieval, routing, and representation learning are jointly optimized.

This work is presented as a conceptual research article. It does not claim empirical validation. Instead, it establishes mathematical scaffolding, explicit propositions, falsification conditions, implementation pathways, and expected failure modes for future experimental work. In doing so, it serves as the first component of a broader research program investigating the internal organization of machine cognition.

Keywords: Differentiable Latent Indexing (DLI); Large Language Models (LLMs); Retrieval-Augmented Generation (RAG); Cognitive Architectures; Neural Retrieval; Differentiable Memory; Transformer Models; Cognitive Complete AI; Hypothesis-Extension Research; Artificial Intelligence Theory.

Position within the Research Program

This manuscript constitutes Volume I of *The Latent Cognition Trilogy*. It develops the memory layer of the broader research program by formalizing Differentiable Latent Indexing (DLI) as a candidate mechanism for retrieval-native cognition inside Transformer-based systems.

The closely related manuscript *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*, submitted to the *Journal of Artificial Intelligence and Knowledge Engineering*, provides the formal architectural foundation for this volume. The present text adapts and extends that foundation within the larger Trilogy sequence, where memory, symbolic stabilization, and hierarchical routing are treated as interdependent layers of latent cognition.

Accordingly, Volume I should not be read as a companion hypothesis paper. It is the opening movement of the Trilogy. The companion hypothesis papers occupy a separate position: they isolate specific claims that support, constrain, or extend the main sequence without replacing it.

Chapter 2

Memory Fields and Differentiable Latent Indexing

Proposition 1: Retrieval Efficiency Gain

If DLI provides a meaningful internal memory mechanism, models augmented with DLI should demonstrate improved performance on tasks requiring long-range dependency resolution compared with baseline transformers of equivalent parameter count.

Falsification condition: If no statistically significant improvement is observed across controlled benchmarks, the hypothesis that internally differentiable retrieval improves effective memory capacity is weakened.

Proposition 2: Latent Clustering Stability

DLI should induce measurable clustering in latent representations corresponding to recurrent semantic structures.

Falsification condition: If latent representations remain uniformly distributed or indistinguishable from baseline attention patterns, the claim of structured internal memory is not supported. To mathematically verify this organization, the spatial distribution of the latent memory field can be monitored via a localized Silhouette Coefficient (S). For a cluster of recurrent semantic structures C, the stability metric is formalized as:

$$S(M^{(\ell)}) = \frac{1}{|C|} \sum_{i \in C} \frac{b(i) - a(i)}{\max(a(i), b(i))}, \tag{2.1}$$

where $a(i)$ represents the mean intra-cluster distance of representation i, and $b(i)$ represents the mean nearest-cluster distance to any other semantic partition. The proposition holds if and only if $S(M^{(\ell)}) > \epsilon$ as training progress approaches convergence, where ϵ is the baseline uniform distribution threshold.

Proposition 3: Retrieval-Induced Drift

Repeated retrieval operations should introduce measurable drift in latent state trajectories.
Falsification condition: If no measurable deviation from baseline forward-pass dynamics is observed, the proposed retrieval interaction is not functionally distinct.

Introduction

The present volume inaugurates *The Latent Cognition Trilogy*, a sequence of conceptual research works centered on latent cognition in artificial systems. Volume I formalizes *Differentiable Latent Indexing* (DLI) as a native retrieval mechanism inside Transformer-based language models. It does not report experimental results; rather, it formalizes, elaborates, and critically develops a theoretical architecture in which memory retrieval becomes part of the model's differentiable computational pathway.
The paper extends the **Differentiable Latent Indexing inside Large Language Models (DLI-LLM)** hypothesis by positioning it as an architectural paradigm for retrieval inside Transformer computation.
Volume II examines the conditions under which compressed latent representations acquire symbolic stability under curriculum, compression, and structural constraint. **Volume III** extends the sequence by analyzing hierarchical routing as a mechanism of architectural coordination across specialized computational components. Together, these papers aim to establish a conceptual lineage for AI systems that treats memory, routing, representation, and control as interdependent components of machine cognition.

This article was developed in response to a hypothesis generated by the Academia.edu AI-assisted scientific modeling platform. The original hypothesis proposed a learnable, differentiable memory-retrieval mechanism embedded directly within Transformer layers. Rather than treating that hypothesis as a fixed endpoint, the present paper treats it as a conceptual seed: a starting point for examining the architectural, cognitive, mathematical, and safety implications of internalized retrieval.

The problem space addressed here is the separation between parametric memory, stored within model weights, and non-parametric memory, retrieved through external documents, embedding stores, or vector databases. Retrieval-augmented generation has improved factual grounding by combining sequence-to-sequence generation with external dense retrieval [17]. Related approaches, including nearest-neighbor language models and memorizing transformers, demonstrate that language models can benefit from access to stored representations beyond the immediate context window [13, 27]. However, these approaches also preserve a structural separation between the model's internal computation and the retrieval substrate. This separation produces three persistent architectural constraints:

1. latency and bandwidth trade-offs between inference and retrieval operations;

2. semantic drift between retrieved information and the evolving internal context of the model; and

3. limited gradient flow between evidence selection, memory access, and downstream reasoning.

DLI proposes a departure from this pattern. It introduces a differentiable memory module embedded within Transformer layers and trained jointly with the model. Memory queries are formulated from hidden states, routed to a latent key-value memory, and fused back into the model via gated integration. Product quantization provides a possible mechanism for scalable approximate search, while gating regulates when retrieved memory should influence the hidden state [10, 12, 16].

By situating retrieval as an internalized cognitive function, DLI reframes memory access as part of the model's own computational graph rather than as an external service call. The resulting architecture is not merely an optimization of retrieval-augmented generation. It is a theoretical proposal to unify attention, memory, and representational control within Transformer-based systems.

Contributions

This work contributes the following elements:

1. a unified architectural hypothesis that integrates latent memory directly into Transformer computation;

2. a mathematical definition of differentiable retrieval, routing, and gated fusion;

3. a conceptual implementation blueprint for internal retrieval during inference;

4. a comparative analysis connecting DLI to retrieval-augmented generation, kNN language models, differentiable memory systems, recurrent memory transformers, and cognitive architectures; and

5. a foundation for the broader research program developed in the *Latent Cognition Trilogy*.

2.1 Overview of the Paper

Section 2 develops the methodological foundation by decomposing the originating DLI-LLM hypothesis into its constituent architectural claims. Section 3 presents theoretical predictions and falsifiable implications. Section 4 develops the mathematical formulation of Differentiable Latent Indexing. Section 5 provides a conceptual implementation blueprint and comparative literature analysis. Section 6 discusses broader implications for cognitive modeling, learning dynamics, safety, and cognitive-complete Transformer architectures.

2.2 Theoretical Construction and Implications

This paper does not report empirical results. Its contribution is the theoretical development of a Transformer-based architecture through structured hypothesis modeling. The starting point is the DLI-LLM hypothesis, which is decomposed into a set of architectural claims and then expanded into a formal framework suitable for future empirical study.

2.2.1 Hypothesis Decomposition

The original hypothesis can be decomposed into four architectural claims:

1. a differentiable memory module embedded inside Transformer layers;
2. a product-quantized memory structure for scalable approximate retrieval;
3. a shared latent vector space for token-level representations and document-level embeddings; and
4. a learned query-gating mechanism for retrieval triggering and fusion control.

Each claim has precedent in adjacent literature but differs in the proposed integration. Neural Turing Machines and Differentiable Neural Computers introduced differentiable access to external memory [8, 9]. Memory-augmented neural networks demonstrated rapid assimilation of new examples through content-based memory access [21]. Recurrent memory transformers and memorizing transformers later adapted memory mechanisms to Transformer-based language modeling [3, 27]. DLI differs from these systems in that it treats latent retrieval as an internal, trainable component of the Transformer layer itself.

2.2.2 Architectural Modeling

The hypothesis is extended into a multi-stage retrieval loop inside the forward pass. At a high level, the current hidden state is used to generate a query vector. That query vector retrieves top-k value vectors from a latent key–value memory. The retrieved representation is then fused back into the model through a learned routing gate. This loop creates a differentiable pathway between hidden-state evolution and memory access.
The proposed retrieval loop contains three major operations:

1. cross-layer memory interpolation, in which retrieval results influence subsequent Transformer layers;
2. sparse top-k selection, allowing retrieval to remain computationally bounded; and
3. internal–external evidence arbitration, in which latent memory competes with or complements the local attention pathway.

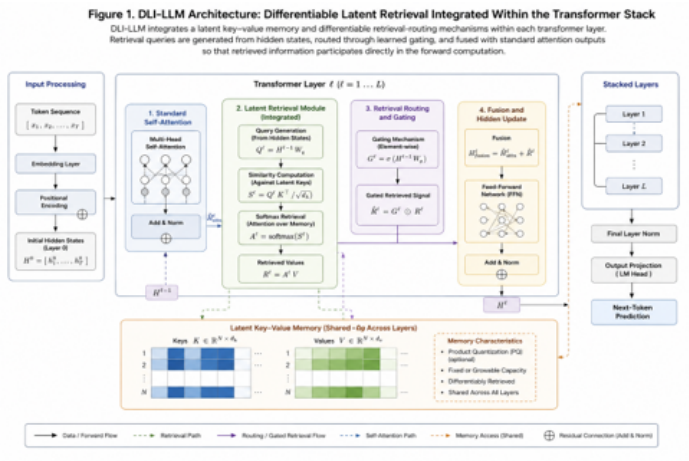

Figure 2.1: Differentiable Latent Indexing architecture. The diagram illustrates how latent key–value memory slots are queried from the current hidden state and fused back into the Transformer attention pathway during the forward pass.

To visualize the core concept, **Figure** 2.1 introduces the proposed DLI-LLM architecture.

2.2.3 Comparative Contextualization

DLI intersects with several existing research lines. Retrieval-augmented generation combines parametric sequence generation with non-parametric document retrieval [17]. kNN-LM interpolates language-model predictions with nearest-neighbor lookup over stored representations [13]. Memorizing Transformers store recent key–value representations for later lookup [27]. Dense passage retrieval systems optimize learned representations for open-domain question answering [19]. Memory-augmented neural networks and differentiable memory systems demonstrate that memory can be made trainable through differentiable addressing [8, 9, 21].

DLI draws on all of these traditions but shifts the point of integration. Rather than positioning retrieval as a pre-generation evidence step or a post-hoc interpolation mechanism, DLI treats retrieval as part of the Transformer layer's representational dynamics. The implication is architectural: the model does not merely consult memory; it computes through memory.

2.3 Theoretical Predictions

Because no DLI-LLM model is implemented in this article, the following claims should be read as theoretical predictions rather than empirical results. They define expected behaviors that future experimental studies should attempt to confirm, refine, or falsify.

- **Retrieval latency.** By integrating retrieval into the forward pass, DLI-LLM is expected to reduce the external retriever bottleneck characteristic of conventional RAG architectures. Approximate nearest-neighbor methods and GPU-accelerated similarity search provide a plausible basis for this efficiency claim [10, 12].
- **Grounding stability.** Stable latent memory may reduce hallucination in knowledge-intensive tasks by narrowing the distance between retrieved evidence and internal representation. This prediction is motivated by improvements observed in retrieval-augmented generation and nearest-neighbor language modeling [17, 13].
- **Generalization through memory retention.** Because latent memory slots are differentiably integrated into Transformer layers, DLI may reproduce some of the generalization-through-memorization dynamics observed in kNN-LM and memorizing transformers [13, 27].
- **Few-shot adaptation.** Learned gating may improve adaptation under limited supervision by controlling when retrieved memory should alter the hidden state, a behavior related to memory-augmented few-shot learning [21].
- **Cross-layer coherence.** Query propagation across layers may improve long-context coherence by maintaining retrieval continuity over successive transformations, complementing trends in recurrent memory transformers [3].

The predicted failure modes are equally important:

- *Memory saturation*: fixed-size latent memory may become overloaded under high document or representation density [8, 27].
- *Value-vector redundancy*: top-k retrieval in high-dimensional latent spaces may return semantically redundant vectors, reducing effective

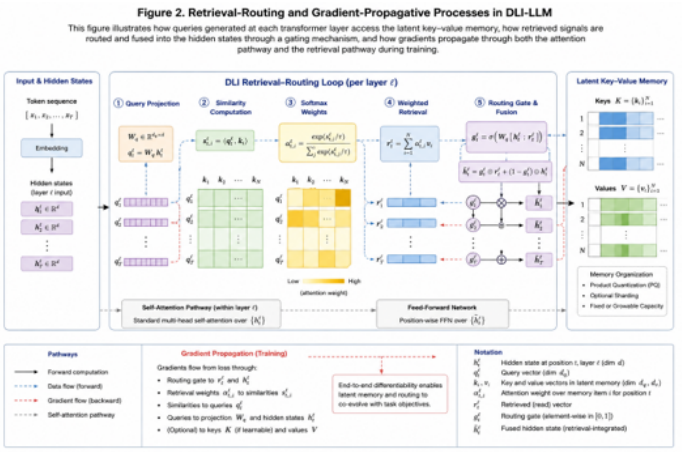

Figure 2.2: Standard retrieval-augmented generation pipeline. In conventional RAG, retrieval remains externally mediated through a document index or vector store before generation. DLI proposes to move part of this retrieval logic into the model's differentiable computation path.

retrieval precision without pruning, compression, or quantization [10, 12].

- *Context misalignment*: retrieval queries may drift when cross-layer propagation interacts poorly with earlier representations, particularly in recurrent or persistent-memory settings [3].
- *Opacity of internal retrieval*: once retrieval is internalized, the evidentiary path may become harder to inspect than in externally mediated RAG systems.

Figure 2.2 contrasts the conventional retrieval-augmented generation pattern with the internalized retrieval logic proposed by DLI.

2.4 Mathematical Framing of Differentiable Latent Indexing

To transition from conceptual architecture to a formally analyzable model, this section expresses DLI-LLM as a minimal extension of standard Transformer notation. Let $h_t^{(\ell)} \in \mathbb{R}^d$ denote the hidden state at token position t and layer ℓ, following the general structure of attention-based Transformer

computation [26]. DLI augments this computation with a latent key–value memory and a differentiable routing mechanism.

2.4.1 Latent Key–Value Memory

For each layer ℓ, define a latent memory bank:

$$M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^{N}, \tag{2.2}$$

where $k_i^{(\ell)} \in \mathbb{R}^{d_k}$ is a memory key and $v_i^{(\ell)} \in \mathbb{R}^{d_v}$ is the corresponding memory value. The memory bank may contain internal token-derived representations, document-derived embeddings, or a hybrid of both. This formulation extends differentiable memory systems by embedding memory within layer-level computation rather than attaching it as a separate controller [8, 9, 21].

2.4.2 Query Projection

A learned projection maps the current hidden state into a retrieval query:

$$q_t^{(\ell)} = W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}. \tag{2.3}$$

This query projection is analogous to attention-based addressing but is directed toward persistent latent memory rather than only toward positions within the active context window [1, 26].

2.4.3 Similarity and Sparse Retrieval

The model computes retrieval logits through a similarity function:

$$s_i^{(\ell)} = \frac{q_t^{(\ell)} \cdot k_i^{(\ell)}}{\sqrt{d_k}} \tag{2.4}$$

A sparse top-k selection operator restricts retrieval to the most relevant memory entries:

$$\alpha_i^{(\ell)} = \frac{\exp(s_i^{(\ell)}/\tau)\mathbf{1}[i \in \mathrm{TopK}(s^{(\ell)}, k)]}{\sum_{j \in \mathrm{TopK}(s^{(\ell)}, k)} \exp(s_j^{(\ell)}/\tau)} \tag{2.5}$$

where τ is a temperature parameter and $\alpha_i^{(\ell)}$ is the differentiable retrieval weight assigned to memory slot i. Product quantization may be used to approximate key search at scale [10, 12].

2.4.4 Retrieved Representation

The retrieved latent representation is computed as a weighted sum of memory values:

$$r_t^{(\ell)} = \sum_{i=1}^{N} \alpha_i^{(\ell)} v_i^{(\ell)}. \tag{2.6}$$

This operation turns memory access into a differentiable aggregation step, preserving gradient flow through the retrieval distribution.

2.4.5 Gated Fusion

A learned gate controls the influence of retrieved memory on the evolving hidden state:

$$g_t^{(\ell)} = \sigma\left(W_g^{(\ell)}[h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)}\right), \tag{2.7}$$

where $[h_t^{(\ell)} \parallel r_t^{(\ell)}]$ denotes vector concatenation and σ is the logistic sigmoid. The fused representation is then:

$$\tilde{h}_t^{(\ell)} = \text{LayerNorm}\left(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)}\right). \tag{2.8}$$

This gating operation connects DLI to conditional computation and mixture-style routing, while preserving a layer-local mechanism for regulating memory influence [16, 3].

2.4.6 Training Objective

A conceptual DLI training objective can be written as:

$$\mathcal{L}_{DLI} = \mathcal{L}_{LM} + \lambda_s \mathcal{L}_{sparse} + \lambda_m \mathcal{L}_{memory} + \lambda_g \mathcal{L}_{gate} \tag{2.9}$$

where $\mathcal{L}_{LM}$ is the standard language modeling loss, $\mathcal{L}_{sparse}$ regularizes retrieval selectivity by penalizing the entropy of the allocation profile α, $\mathcal{L}_{memory}$ regulates memory coherence or redundancy, and $\mathcal{L}_{gate}$ discourages uncontrolled reliance on retrieved values. The exact form of these terms is an empirical design question, but the formulation clarifies where future implementations may intervene.

2.4.7 Gradient Flow and Optimization Continuity

Because the sparse selection operator $\mathrm{TopK}(\cdot)$ introduces a non-differentiable boundary, gradient continuity through the retrieval pathway is maintained via the soft-max weighting allocation across the chosen partition. The partial derivative of the fused representation $\tilde{h}_t^{(\ell)}$ with respect to the memory keys $k_i^{(\ell)}$ propagates directly through the similarity scaling:

$$\frac{\partial \tilde{h}_t^{(\ell)}}{\partial k_i^{(\ell)}} = g_t^{(\ell)} \odot v_i^{(\ell)} \left(\frac{\partial \alpha_i^{(\ell)}}{\partial s_i^{(\ell)}} \right) \left(\frac{q_t^{(\ell)}}{\sqrt{d_k}} \right). \tag{2.10}$$

This formalizes a continuous, uninterrupted backward pass from downstream language modeling errors directly into the latent memory coordinates, allowing the index structure to co-evolve with parametric transformations without a separate optimization loop.

Figure 2.3 illustrates how gradients propagate through the attention and retrieval pathways during training.

2.5 Algorithmic Specification and Implementation Blueprint

To operationalize the retrieval mechanism described above, Algorithm 2.5 specifies a single DLI retrieval step within one Transformer layer.

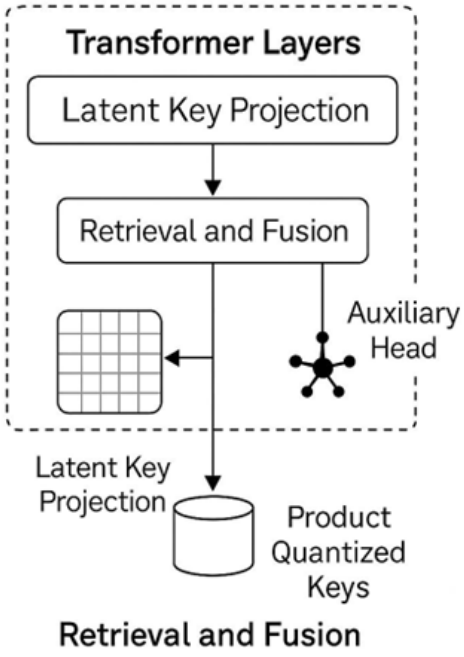

Figure 2.3: Gradient pathways in Differentiable Latent Indexing. Query projection, similarity computation, sparse retrieval weighting, routed memory integration, and gated fusion remain within the differentiable computation graph, allowing latent memory to co-evolve with Transformer parameters.

Algorithm 1: Differentiable Latent Indexing Loop
Input: hidden state $h_t^{(\ell)}$, latent memory $M^{(\ell)} = \{(k_i^{(\ell)}, v_i^{(\ell)})\}_{i=1}^{N}$, parameters $W_q^{(\ell)}$, $W_g^{(\ell)}$, top-k value, temperature τ.
Output: fused representation $\tilde{h}_t^{(\ell)}$.

1. Compute retrieval query: $q_t^{(\ell)} \leftarrow W_q^{(\ell)} h_t^{(\ell)} + b_q^{(\ell)}$.
2. Compute similarity scores: $s_i^{(\ell)} \leftarrow q_t^{(\ell)} \cdot k_i^{(\ell)} / \sqrt{d_k}$.
3. Select top-k memory slots according to $s_i^{(\ell)}$.
4. Compute retrieval weights: $\alpha^{(\ell)} \leftarrow \text{softmax}(s^{(\ell)}/\tau)$ over selected slots.
5. Retrieve latent value: $r_t^{(\ell)} \leftarrow \sum_i \alpha_i^{(\ell)} v_i^{(\ell)}$.
6. Compute routing gate: $g_t^{(\ell)} \leftarrow \sigma(W_g^{(\ell)}[h_t^{(\ell)} \parallel r_t^{(\ell)}] + b_g^{(\ell)})$.
7. Fuse retrieved memory with hidden state: $\tilde{h}_t^{(\ell)} \leftarrow \text{LayerNorm}(h_t^{(\ell)} + g_t^{(\ell)} \odot r_t^{(\ell)})$.
8. Return $\tilde{h}_t^{(\ell)}$.

Although this paper is theoretical, an implementable DLI prototype would require four design stages.

2.5.1 Memory Construction

Document embeddings, token representations, or intermediate hidden states would be transformed into latent key–value banks. Product quantization could then be applied to compress memory and enable approximate retrieval at scale [10, 12].

2.5.2 Quantized Latent Indexing Mechanics

To prevent the TopK operations from scaling linearly $\mathcal{O}(N)$ with the size of the latent memory repository, the key space is structured using Product Quantization (PQ). The high-dimensional key space $\mathbb{R}^{d_k}$ is decomposed into M orthogonal subspaces:

$$\mathbb{R}^{d_k} = \mathbb{R}^{d_s} \times \mathbb{R}^{d_s} \times \cdots \times \mathbb{R}^{d_s}, \tag{2.11}$$

where $d_s = d_k/M$. Each subspace is quantized into K^* centroids using a learned codebook. During the forward pass, the query projection $q_t^{(\ell)}$ is similarly sliced into M sub-vectors. Similarity estimation $s_i^{(\ell)}$ is then executed via asymmetric distance computation (ADC) using pre-calculated look-up tables. This reduces the computational overhead of the internalized retrieval phase to a collection of parallel cache lookups, maintaining a highly bounded inference budget.

2.5.3 Layer Integration

A DLI block would be inserted after self-attention or between attention and feed-forward sublayers. The routing gate would determine whether retrieved values should influence the hidden state. This placement allows DLI to function as a memory-aware extension of Transformer computation rather than as a separate retrieval pipeline.

2.5.4 Training and Fine-Tuning

Training would jointly optimize language modeling, retrieval selectivity, memory coherence, and gating behavior. In a pre-training setting, memory slots could evolve with the model. In a fine-tuning setting, selected memory banks could be frozen, swapped, or domain-adapted.

2.5.5 Inference Behavior

During inference, DLI could operate through frozen or partially updateable memory. Sparse top-k retrieval would limit computational overhead. Memory swapping could support domain-specific deployment, while gating thresholds could regulate when latent memory influences generation.

2.5.6 Evaluation Plan

A future empirical study should compare DLI-augmented models against baseline Transformers, RAG models, kNN-LM variants, and memorizing transformers. Evaluation should include latency overhead, retrieval precision, hallucination rates, long-context coherence, memory saturation, and ablation studies that independently disable routing, memory access, or quantization.

2.6 Comparative Literature Analysis

Table 2.1 summarizes the position of DLI relative to adjacent architectures.

2.7 Discussion

The DLI-LLM architecture reframes how memory is conceptualized in Transformer-based systems. By integrating retrieval into the forward pass, the model collapses the traditional separation between reasoning and recall. This aligns with cognitive architecture research that treats memory, control, learning, and representation as interdependent properties of intelligent systems [23].

2.7.1 Cognitive Implications

Embedding differentiable latent memory directly within Transformer layers positions retrieval as a native cognitive operation rather than an external API call. This shift parallels earlier insights from memory-augmented neural networks and differentiable memory systems [8, 9, 21], but extends them by proposing a unified latent space where token-derived representations and document-derived vectors can coexist. The resulting architecture moves toward cognitive completeness by enabling context-dependent recall, internal evidence arbitration, and persistent cross-layer memory flow.

2.7.2 Learning Dynamics and Adaptation

DLI-LLM suggests new possibilities for lifelong and continual learning. If memory slots can be updated incrementally, the model may integrate new documents without full retraining, a capability related to recurrent memory systems and neural semantic encoders [3, 18]. However, this advantage introduces challenges. Latent memory may overfit internal distributions, memory slots may saturate, and retrieval may become redundant without pruning, quantization, or regularization [10, 12].

2.7.3 Bias, Safety, and Privacy Considerations

The integration of retrieval into model computation raises safety concerns. Frozen memory layers may preserve sensitive information, creating privacy risks. Internal retrieval may reinforce biased representations. Latent memories, once implicitly learned, may be harder to inspect than externally

Architecture	**Relation to DLI**	**Limitation Relative to DLI**
Retrieval-Augmented Generation	Combines parametric generation with non-parametric retrieval [17].	Retrieval is typically external to the Transformer computation graph.
kNN-LM	Interpolates language-model predictions with nearest-neighbor lookup [13].	Retrieval is post-hoc and not trained as an internal layer function.
Memorizing Transformers	Store and retrieve recent key–value representations [27].	Memory improves context use but remains distinct from document-level latent unification.
Neural Turing Machines / DNC	Demonstrate differentiable memory addressing [8, 9].	Designed as general memory controllers rather than layer-native LLM retrieval modules.
Memory-Augmented Neural Networks	Support rapid assimilation of new examples [21].	Primarily oriented toward few-shot learning rather than Transformer-internal retrieval.
Recurrent Memory Transformer	Uses memory tokens for long-sequence processing [3].	Memory persistence is present, but explicit latent key–value retrieval control is limited.
Dense Passage Retrieval / RocketQA	Optimizes learned representations for retrieval [19].	Retrieval occurs outside the model's internal forward computation.
Product-Quantized Vector Search	Provides scalable approximate search [10, 12].	Supplies retrieval infrastructure but not a cognitive or Transformer-layer architecture.

Table 2.1: Comparative analysis of memory-augmented and retrieval-integrated architectures.

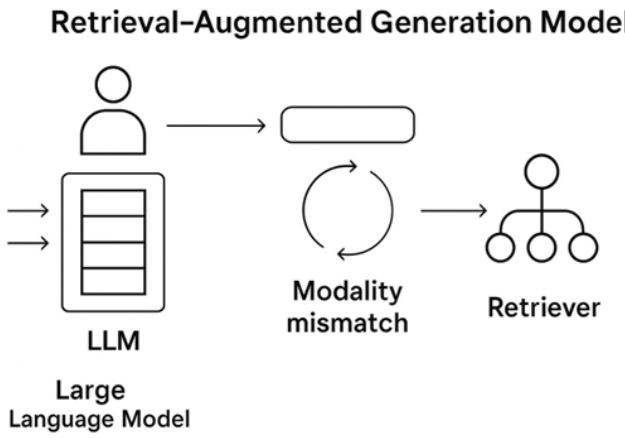

Figure 2.4: Training flow and gating control in DLI-LLM. Product-quantized key–value lookup, attention fusion, routing gates, and gradient-based memory adaptation interact across Transformer blocks.

retrieved documents. These risks parallel issues in retrieval-augmented systems but are amplified by the opacity of internalized memory [17].

2.7.4 Architectural Control and Gating Mechanisms

A central feature of DLI-LLM is the routing gate. The gate mediates the integration of retrieved values into the hidden state, determines how retrieval influences inference, and regulates cross-layer propagation. Conceptually, this places gating at the core of cognitive control: the architecture must learn not only what to retrieve but also when retrieval matters. **Figure** 2.4 illustrates this control path.

2.7.5 Toward Cognitive-Complete Transformer Architectures

DLI-LLM argues for reconsidering memory-augmented Transformers not as storage-bound enhancements, but as components of a broader approach to machine cognition. In this view, retrieval is not merely a way to extend context length; it is a candidate mechanism for internal evidence selection, representational persistence, and adaptive control. This aligns with the broader desideratum that cognitive architectures should integrate memory, learning, representation, and control into a coherent system [23].

2.8 Conclusion

Differentiable Latent Indexing proposes that retrieval can be moved from the periphery of language-model architecture into the differentiable interior of Transformer computation. The framework introduced here formalizes this idea through latent key–value memory, query projection, sparse retrieval, gated fusion, and a conceptual training objective. It also defines falsifiable propositions, expected benefits, and plausible failure modes.
The paper does not claim that DLI-LLM has already been implemented or empirically validated. Its contribution is theoretical: it clarifies what such an architecture would require, how it differs from existing retrieval and memory-augmented models, and what future experiments would need to test. As the first volume in the *Latent Cognition Trilogy*, this article establishes memory as the initial boundary condition for a broader investigation into artificial cognition.

Volume II: Symbol Fields

Curriculum-Induced Symbol Invention for Emergent Reasoning Primitives

Abstract

This article introduces *Curriculum-Induced Symbol Invention* (CISI), a theoretical framework for examining how discrete reasoning primitives may emerge within the intermediate representational pathways of Large Language Models (LLMs). Standard Transformer architectures operate primarily over continuous hidden states, and while these states may support complex behavior, the mechanisms by which recurring computational patterns stabilize into reusable abstractions remain only partially formalized. CISI proposes an architectural modification in which hidden-state trajectories are intercepted and compressed through an auxiliary vector-quantized latent codebook layer during pre-training.

Driven by a staged curriculum of increasing task complexity and an informational bottleneck, the model is pressured to stabilize and reuse internal codewords that function as compact computational primitives. In this sense, CISI treats symbol formation not as a post hoc linguistic mapping, but as a structural response to compression, reuse, and representational economy.

Building upon the internalized memory substrate established in *Volume I: Memory Fields*, this work shifts the focus from continuous storage and retrieval to discrete abstraction. It provides a self-contained theoretical specification, including training stages, codebook optimization dynamics, semiographic translation probes, formal propositions, falsification conditions, and empirical evaluation protocols for the study of semiotics.

Keywords: Curriculum-Induced Symbol Invention (CISI); Latent Semiography; Vector Quantization; Information Bottleneck; Domain-Specific Languages; Machine Cognition; Structural Representation Learning; Automated Abstraction.

Position within the Research Program

This manuscript constitutes **Volume** II of *The Latent Cognition Trilogy*. It develops the symbolic abstraction layer of the broader research program by formalizing *Curriculum-Induced Symbol Invention* (CISI) as a candidate mechanism through which compressed latent representations may acquire symbolic stability.

Volume I, *Memory Fields*, established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational structure. **Volume II** shifts the focus from memory access to symbolic compression. Its central claim is that, under curriculum pressure and representational bottlenecks, neural systems may stabilize reusable internal primitives that function as machine-generated symbolic structures.

The architectural foundation for the broader program remains *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*. That work supplies the retrieval-native substrate. The present volume extends the sequence by asking how retrieved, compressed, and reused latent structures may become symbolically stable.

Accordingly, **Volume II** should not be read as a companion hypothesis paper. It is the second movement of the Trilogy. The companion hypothesis papers occupy a separate role: they isolate supporting claims without replacing the main sequence.

Chapter 3

Symbol Fields and Curriculum-Induced Symbol Invention

3.1 Introduction

Volume II of *The Latent Cognition Trilogy* examines the transition from latent memory to symbolic stabilization. Volume I established memory as a structured field of retrievable latent representations. The present volume asks a different question: under what conditions can compressed internal representations acquire sufficient stability, reusability, and functional specificity to serve as symbolic primitives within a neural architecture? The central construct introduced here is *Curriculum-Induced Symbol Invention* (CISI). CISI proposes that symbolic structure need not be imposed externally through hand-coded rules, symbolic grammars, or explicit logical systems. Instead, symbolic-like structures may emerge when a model is forced to solve increasingly complex tasks through a constrained intermediate representational bottleneck. Under such conditions, recurring computational patterns may be compressed into reusable latent codewords that function as internal abstractions. The argument is not that large language models already possess explicit symbolic languages in the classical sense. Nor does CISI claim that vector quantization alone produces reasoning. The claim is narrower: a discrete intermediate codebook, trained under curriculum pressure and evaluated through stability, reuse, and transfer diagnostics, may provide a mechanism for studying how latent representations become symbolically organized.

3.2 Problem Space: From Continuous Representation to Symbolic Stability

Transformer-based language models operate over high-dimensional continuous hidden states. These states support complex linguistic and reasoning behaviors, but their internal organization often remains difficult to characterize. A hidden state may simultaneously encode context, syntax, semantic associations, task-relevant features, and transient computational dynamics. This expressive richness creates a problem for symbolic interpretation: the model may behave as though it has reusable abstractions without exposing clear internal units corresponding to those abstractions. Classical symbolic systems rely on discrete, composable, and reusable primitives. Neural systems, by contrast, distribute information across continuous representations. The challenge is therefore not simply to recover symbols from neural states, but to determine whether neural architectures can be pressured to form stable intermediate structures that play a symbol-like role without abandoning differentiable learning. CISI addresses this problem by introducing three constraints. First, the model is trained under a staged curriculum that progressively increases compositional complexity. Second, an intermediate vector-quantized bottleneck prevents all information from passing through unconstrained continuous channels. Third, diagnostic probes evaluate whether discrete codewords become stable, reusable, and predictive of downstream reasoning operations. Within this framework, symbolic stability is defined operationally rather than metaphysically. A latent codeword becomes symbolically relevant only if it satisfies three conditions: it is selected repeatedly across related contexts, it contributes measurably to task performance, and its functional role remains sufficiently stable across curriculum stages and out-of-distribution evaluations.

3.3 Contributions

This volume contributes the following elements:

1. a theoretical framework for studying symbolic stabilization under curriculum pressure and representational compression;

2. a vector-quantized bottleneck mechanism for forcing intermediate latent states into discrete reusable codewords;

3. a full CISI training objective that combines language modeling, vector quantization, commitment, entropy regulation, reuse pressure, and symbolic stability;
4. diagnostic metrics for codebook utilization, reuse concentration, symbolic drift, and codeword stability;
5. a staged curriculum design intended to induce primitive, compositional, and integrative symbolic behavior;
6. a semiographic probing framework for translating latent codeword activations into interpretable operational roles; and
7. a set of failure modes and falsification conditions for distinguishing genuine symbolic stabilization from arbitrary discretization.

3.4 Overview of the Volume

The remainder of this volume proceeds as follows. The next chapter defines the core propositions and falsification conditions for CISI. The following chapter develops the informational bottleneck and vector quantization mechanism. Subsequent chapters introduce the full CISI objective, codebook diagnostics, staged curriculum design, semiographic interpretation, failure modes, empirical evaluation protocols, and the relationship between symbolic stabilization and the routing mechanisms developed in Volume III.

Chapter 4

Related Work

4.1 Discrete Latent Representations

CISI is closely related to research on discrete latent representation learning. Vector-quantized latent models demonstrate that continuous inputs can be mapped into learned discrete codebooks while preserving differentiability through surrogate gradient mechanisms [25]. This provides an important precedent for the CISI bottleneck: the architecture does not merely compress representations, but forces them to pass through a finite set of reusable latent units. However, CISI differs from standard vector-quantized representation learning in its intended role. The codebook is not treated only as a compression device or generative modeling component. It is treated as a candidate substrate for symbolic stabilization, where codeword reuse, curriculum exposure, and downstream reasoning performance jointly determine whether a codeword has acquired functional symbolic value.

4.2 Information Bottleneck and Compression

The information bottleneck framework provides a theoretical basis for treating compression as a mechanism for extracting task-relevant structure [24]. In CISI, the bottleneck is not only informational but semiotic: it pressures the model to preserve useful computational distinctions while discarding irrelevant representational variation. A successful CISI codebook should therefore retain information needed for reasoning while reducing dependence on unconstrained continuous state propagation. This distinction is important because not all compression is symbolic. A repre-

sentation may be compressed while remaining uninterpretable, unstable, or task-specific. CISI therefore requires additional diagnostics for reuse, stability, and transfer beyond reconstruction error alone.

4.3 Curriculum Learning and Compositional Generalization

Curriculum learning suggests that training order can shape the acquisition of increasingly complex behaviors [2]. CISI adopts this principle by exposing the model to progressively structured tasks: primitive operations, compositional routines, and integrative reasoning contexts. The curriculum is not merely a training convenience. It functions as the developmental pressure under which reusable internal codewords may become stabilized. This connects CISI to the broader problem of compositional generalization. Studies of systematic generalization show that neural models may struggle to transfer learned operations compositionally when training distributions do not force reusable abstractions [15]. CISI addresses this limitation by treating compositional transfer as a test of whether latent codewords have become functionally reusable rather than locally memorized.

4.4 Emergent Communication and Latent Semiotics

Research on emergent communication shows that agents can develop task-specific signaling systems under pressure for coordination and efficiency [6]. CISI adapts this insight to the internal dynamics of a single model. Instead of agents communicating with one another, intermediate layers communicate through a constrained latent codebook. The resulting codewords may be interpreted as a form of internal semiography: a machine-generated representational scheme optimized for task performance rather than human readability. This analogy must be used carefully. CISI does not assume that codewords are linguistic symbols. It treats them as operational primitives whose symbolic status must be earned through stability, reuse, transfer, and interpretability tests.

4.5 Probing and Interpretability

The interpretability of internal representations remains an open challenge in large language models. Probing studies attempt to identify what linguistic, semantic, or structural information is encoded in learned representations [20]. CISI extends this approach by making the intermediate representational substrate partially discrete. This may improve inspectability, but it also introduces a risk: discrete codewords can invite overinterpretation. For this reason, CISI does not equate probe labels with symbol meaning. A codeword is not considered symbolic simply because a probe can attach a human-readable label to it. The label must remain stable across contexts, correspond to measurable functional effects, and survive ablation tests that demonstrate its contribution to downstream reasoning.

Chapter 5

Propositions and Falsification Conditions

The validity of Curriculum-Induced Symbol Invention as a theoretical mechanism rests upon explicit, predictable behavioral deviations from standard Transformer baselines. The following formal propositions establish the empirical boundaries of the architecture and dictate clear falsification parameters.

Proposition 1: Symbolic Abstraction Efficiency

As structural curriculum complexity scales, an LLM computational graph constrained by a discrete intermediate vector-quantization bottleneck will demonstrate significantly greater cross-domain generalization and sample efficiency on out-of-distribution (OOD) reasoning tasks than standard continuous-state architectures with equivalent parameter density.

Falsification Condition: If a continuous-state baseline matches or exceeds the CISI architecture on downstream multi-step reasoning evaluations when controlled for training token exposure and active parameter count, the claim that discrete intermediate symbol coining resolves structural compositionality is falsified.

Proposition 2: Codebook Reuse and Structural Sparsity

The optimization trajectory of a bounded latent codebook operating under variable curriculum demands will favor stabilizing a minimal, highly reusable set of primitive codewords over a strategy of continuous, uniform expansion of codebook resources.
Falsification Condition: If tracking codebook utilization profiles reveals a linear growth path proportional to task variation, or if the codebook selection entropy approaches maximum uniform distribution without distinct topological clustering, the hypothesis that the informational bottleneck forces symbolic compression is ungrounded.

Proposition 3: Degradation under Bottleneck Deactivation

The deactivation or continuous relaxation of the intermediate quantization layer following curriculum exposure will result in a catastrophic degradation of out-of-distribution reasoning capabilities, while local in-distribution language modeling performance remains unaffected.
Falsification Condition: If an operational CISI model maintains its advanced multi-step reasoning capabilities when the vector quantization layer is bypassed or replaced with an unconstrained linear projection, the symbolic utility of the discrete codebook is disproven.

Chapter 6

The Informational Bottleneck and Vector Quantization

Let $h_t^{(\ell)} \in \mathbb{R}^d$ define the continuous hidden representation of a token at sequence position t and intermediate layer ℓ within the standard Transformer forward pass. The CISI architecture systematically intercepts this trajectory, passing the continuous vector field through a discrete quantization bottleneck before it propagates to subsequent layers.

6.1 Vector Quantization Mechanics

We define an internal, learned discrete codebook dictionary $\mathcal{C} = \{e_1, e_2, \ldots, e_K\}$, where each individual codeword vector $e_k \in \mathbb{R}^d$. The continuous latent state $h_t^{(\ell)}$ is mapped directly to its nearest symbolic primitive within the dictionary via a minimum Euclidean distance projection operator:

$$z_t^{(\ell)} = \text{vquant}\left(h_t^{(\ell)}\right) = e_{\hat{k}}, \quad \text{where } \hat{k} = \arg\min_k \|h_t^{(\ell)} - e_k\|_2. \tag{6.1}$$

The selected codebook index $\hat{k}$ serves as the discrete, organic latent symbol invented by the architecture to represent that particular continuous computational sub-manifold.

6.2 The Compression Objective

To enforce intense compression pressure and protect the network against representational collapse or passive passing-through of uncompressed tokens, an auxiliary decoder head $\mathcal{D}$ is introduced. This decoder is tasked with reconstructing the complete, high-dimensional continuous hidden state $h_t^{(\ell)}$ using the quantized discrete vector assignment $z_t^{(\ell)}$:

$$\mathcal{L}_{\text{comp}} = \|\mathcal{D}\left(z_t^{(\ell)}\right) - \text{sg}[h_t^{(\ell)}]\|_2^2 + \beta\|\,\text{sg}[h_t^{(\ell)}] - z_t^{(\ell)}\|_2^2, \tag{6.2}$$

where $\text{sg}[\cdot]$ represents the classic stop-gradient operator required to prevent unstable backward oscillations during joint optimization, and β is a commitment hyperparameter regulating the alignment rate between continuous trajectories and codebook coordinates.

6.3 Full CISI Training Objective

The vector-quantized bottleneck alone does not define the full CISI training process. It specifies how continuous hidden states are discretized, but the broader architecture requires an objective that jointly optimizes language modeling, codebook commitment, symbolic reuse, sparsity, and stability under curriculum pressure. A conceptual CISI objective can be written as:

$$\mathcal{L}_{CISI} = \mathcal{L}_{LM} + \lambda_q\mathcal{L}_{VQ} + \lambda_c\mathcal{L}_{commit} + \lambda_e\mathcal{L}_{entropy} + \lambda_r\mathcal{L}_{reuse} + \lambda_s\mathcal{L}_{stability}. \tag{6.3}$$

Here, $\mathcal{L}_{LM}$ denotes the standard language modeling objective. The term $\mathcal{L}_{VQ}$ captures reconstruction error through the vector-quantized bottleneck, while $\mathcal{L}_{commit}$ encourages continuous hidden states to remain close to their assigned codebook vectors. The entropy term $\mathcal{L}_{entropy}$ regulates the distribution of codeword usage, preventing uncontrolled uniform expansion or premature collapse into a small number of dominant codes. The reuse term $\mathcal{L}_{reuse}$ encourages frequently useful codewords to persist across related tasks, while $\mathcal{L}_{stability}$ penalizes excessive drift in codeword meaning across curriculum stages.

The purpose of this objective is not merely to discretize hidden states. Its purpose is to create pressure for reusable symbolic structure. Under this formulation, a codeword becomes symbolically meaningful only if it is repeatedly selected, functionally useful, stable across related contexts, and predictive of downstream reasoning behavior.

6.4 Codebook Diagnostics and Symbolic Stability

The emergence of symbolic structure cannot be inferred from codebook discreteness alone. A vector-quantized layer may produce discrete indices without producing meaningful symbolic primitives. CISI therefore requires diagnostic criteria for distinguishing genuine symbolic stabilization from arbitrary quantization.

Let $p(e_k)$ denote the empirical activation frequency of codeword e_k over a validation corpus. Codebook utilization entropy may be defined as:

$$H(\mathcal{C}) = -\sum_{k=1}^{K} p(e_k) \log p(e_k). \tag{6.4}$$

A maximally uniform distribution may indicate insufficient specialization, while extreme concentration may indicate codebook collapse. Symbolic stabilization is expected to occur in an intermediate regime where a subset of codewords becomes highly reusable without eliminating functional diversity.

A reuse concentration score can be expressed as:

$$R(\mathcal{C}) = \sum_{k=1}^{K} p(e_k)^2. \tag{6.5}$$

Higher values of $R(\mathcal{C})$ indicate repeated reliance on a smaller set of codewords, but this metric must be interpreted jointly with task performance and out-of-distribution generalization. Reuse alone is not sufficient evidence of symbolic structure; the reused codewords must also correspond to stable computational roles.

To evaluate stability across curriculum stages, let $p_t(e_k)$ and $p_{t+1}(e_k)$ denote codeword activation distributions at successive stages. A symbolic drift measure may be written as:

$$D_{symbol}(t, t+1) = D_{KL}\left(p_t(e_k) \parallel p_{t+1}(e_k)\right). \tag{6.6}$$

A stable symbolic codebook should exhibit decreasing drift after curriculum convergence, while still preserving enough plasticity to adapt to higher-order compositional tasks.

Chapter 7

Staged Curriculum Design

The autonomous invention of logical primitives cannot successfully execute within unstructured, high-entropy text environments. It requires an evolutionary gradient. The CISI training framework uses a multi-tiered curriculum that systematically triggers symbol creation by varying operational complexity.

Curriculum Stage	Operational Complexity	Expected Symbolic Emergence
Stage 1: Primitives	Single-digit arithmetic, basic relational logic statements, atomic parsing scripts.	Coining of baseline primitives (e.g., latent equivalents to functional terms like "ADD", "COMPARE").
Stage 2: Composition	Compound functional routines, multi-hop dependencies, variable tracking loops.	Combination of Stage 1 codewords into complex, integrated computational blocks.
Stage 3: Integration	Knowledge-intensive text, open-domain reasoning benchmarks (GSM8K, CommonsenseQA).	Synthesis of a robust, highly optimized internal Domain-Specific Language (DSL).

Table 7.1: The CISI Staged Training Curriculum Profile.

During Stage 1, a minimal partition of 10 to 20 active codewords suffices for error minimization. As the model transitions to Stages 2 and 3, the codebook is permitted to expand under strict sparsity regularizers, thereby forcing previous symbols to act as structural building blocks for higher-order reasoning routines.

Chapter 8

Semiographic Projection and Interpretation

Once symbols stabilize within the intermediate codebook substrate, they must be rendered inspectable to ensure cognitive accountability and alignment. CISI avoids back-correcting the network via human prose; instead, it reads the machine's internal language directly.

8.0.1 Linear Probing and Linguistic Translation

We introduce a periodic probing schedule using linear classification models applied directly to index activation frequencies over standardized validation partitions:

$$P(T_m \mid e_{\hat{k}}) = \text{softmax}\left(W_p e_{\hat{k}} + b_p\right), \tag{8.1}$$

where T_m represents a human-interpretable linguistic concept or operational token. This translation layer functions as a forensic tool, decoding raw latent cluster identifiers into legible intermediate reasoning trajectories, thereby exposing the inner paths of the machine's deductive logic.

Chapter 9

Expected Failure Modes

CISI is a theoretical mechanism, not a guaranteed path to symbolic reasoning. Its value depends on whether the proposed bottleneck and curriculum pressures produce stable, reusable, and functionally relevant codewords. Several failure modes must therefore be considered explicitly.

9.1 Codebook Collapse

The most direct failure mode is codebook collapse. In this case, the model relies on a small number of dominant codewords regardless of task variation. Although such collapse may appear efficient, it would indicate that the bottleneck has failed to preserve meaningful distinctions among computational operations. Collapse can be detected through low codebook entropy, high reuse concentration, and degraded out-of-distribution performance.

9.2 Symbol Fragmentation

The opposite failure mode is symbol fragmentation. Here, the model creates too many context-specific codewords, preventing reusable abstractions from forming. Fragmentation may yield high reconstruction accuracy while failing to support compositional transfer. A fragmented codebook behaves like a lookup table rather than a symbolic substrate.

9.3 Pseudo-Symbol Formation

A third failure mode is pseudo-symbol formation. The model may produce discrete codewords that appear stable under probing but do not causally contribute to reasoning performance. This failure can occur when probe classifiers identify superficial correlations rather than functional computational roles. Ablation tests are therefore necessary to determine whether codewords are operationally necessary.

9.4 Curriculum Overfitting

CISI depends on staged curriculum pressure. If the curriculum is too narrow, the model may invent codewords that are useful only for the training sequence and fail under distributional shift. Such overfitting would weaken the claim that the codebook captures reusable abstractions. The evaluation protocol must therefore include out-of-distribution and compositional transfer tasks.

9.5 Probe-Induced Overinterpretation

Semiographic projection introduces an interpretive risk. Human-readable labels assigned to codewords may suggest more structure than the model actually possesses. A codeword labeled as *COMPARE*, for example, may not correspond to a stable comparison operation across contexts. CISI therefore treats probe labels as hypotheses, not explanations.

9.6 Brittle Symbolic Transfer

A CISI model may show improved performance on structured reasoning tasks while remaining brittle under mixed-domain, noisy, or adversarial inputs. This would suggest that symbolic stabilization has occurred only within a narrow operational regime. A robust symbolic field should preserve useful abstraction without becoming rigid or domain-bound.

Chapter 10

Empirical Evaluation Protocol

To validate the theoretical assertions of Curriculum-Induced Symbol Invention, any experimental implementation must undergo a rigorous, multi-layered forensic protocol comprising four primary validation operations.

10.1 Zero-Shot Out-of-Distribution Evaluation

The primary benchmark for structural symbol creation is the model's ability to perform zero-shot transfer to entirely novel reasoning datasets. Following exposure to the staged curriculum, the CISI architecture must be evaluated on logic and relational datasets whose underlying semantic distributions share zero overlap with the training inputs. Performance metrics must be compared strictly against baseline Transformer models with identical parameter counts and token exposure.

10.2 Ablation of the Compression Interface

To prove that the discrete codebook is the active engine behind reasoning stability, developers must execute a structural ablation pass. By systematically disabling the compression layer post-training—or dynamically substituting the vector quantization operator with an unconstrained, continuous linear layer—the evaluator must track the exact delta in generalization performance. A corresponding drop in out-of-distribution accuracy confirms the operational necessity of the symbols.

10.3 Topological Clustering and Code-Reuse Metrics

The spatial distribution of the codebook must be mathematically evaluated using the localized Silhouette Coefficient (S) formalized in the canon of Volume I. Evaluators must verify that the continuous hidden states collapse into highly concentrated, discrete geometric regions surrounding active codebook centroids. Furthermore, the codebook utilization profile must exhibit a Power Law distribution, confirming high code reuse across disparate tasks.

10.4 Human Semiotic Mapping and Consistency Studies

Finally, the cognitive validity of the invented internal symbols must be tested through a blind human annotation study. Human evaluators are presented with isolated activation maps for specific codewords, alongside their corresponding linguistic contexts, decoded via the linear-probing translation matrix. Annotators are tasked with mapping these active indices to precise English descriptions or operational rules. The degree of semantic agreement among annotators is quantified using Fleiss' Kappa (κ), which verifies whether the machine's invented symbols correspond to stable, invariant concepts.

Chapter 11

Discussion

11.1 Cognitive Interpretation

CISI reframes symbolic abstraction as a possible consequence of representational compression under structured training pressure. In this view, symbols are not assumed to be externally supplied units. They are treated as stabilized internal codes that emerge when the model must preserve reusable computational structure through a constrained channel. This interpretation occupies a middle ground between continuous representation learning and classical symbolic reasoning. CISI does not reject distributed neural representation, but it argues that some forms of reasoning may require intermediate structures that are more stable, reusable, and inspectable than ordinary hidden states. The codebook provides one possible mechanism for producing such structures.

11.2 Relation to Volume I

Volume I established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational fields. Volume II extends that argument by asking how retrieved or compressed latent structures may become symbolically stable. Memory supplies access to latent structure; symbolic stabilization supplies reusable abstraction. The relation between DLI and CISI is therefore sequential but not strictly causal. A system may implement latent memory without symbolic stabilization, and it may implement a symbolic bottleneck without DLI. Within the Trilogy, however, the two mechanisms are treated as complementary layers of a broader latent cognitive architecture.

11.3 Interpretability and Accountability

The partial discreteness of CISI may improve interpretability by creating identifiable intermediate units. However, discreteness alone does not guarantee transparency. A codeword may be stable but uninterpretable, interpretable but non-causal, or causal only within narrow contexts. For this reason, CISI requires multiple forms of evidence: probing, ablation, transfer testing, entropy analysis, and human annotation. This layered interpretability strategy is central to the framework. It prevents the model's invented codebook from being romanticized as a private language unless its operational role can be demonstrated.

11.4 Limits of the Framework

CISI does not prove that neural models can invent symbols in the strong philosophical sense. It defines a testable architectural pathway for studying symbol-like stabilization. The framework remains theoretical until implemented and compared against continuous-state baselines, vector-quantized controls, and curriculum-free alternatives. The strongest version of CISI would be supported only if discrete codewords improve systematic generalization, remain stable across curriculum stages, survive ablation tests, and correspond to interpretable computational roles. Without such evidence, the framework should be treated as a useful compression architecture rather than a theory of machine symbol formation.

Chapter 12

Conclusion

Volume II has developed *Curriculum-Induced Symbol Invention* as the symbolic abstraction layer of *The Latent Cognition Trilogy*. Its central claim is that symbolic stability may arise when continuous latent representations are forced through a discrete bottleneck under curriculum pressure and then reused across increasingly complex tasks. The proposed framework does not equate discretization with symbolism. Instead, it defines symbolic relevance in terms of stability, reuse, functional contribution, and transfer. A codeword becomes symbolically meaningful only when it behaves as a reusable computational primitive rather than as an arbitrary index in a learned codebook. CISI therefore contributes a theoretical bridge between latent memory and architectural routing. Volume I described how memory may be treated as an internal field of retrievable representations. Volume II has examined how such representations may become compressed into reusable abstractions. Volume III extends the sequence by asking how memory fields and symbol fields can be coordinated through hierarchical routing mechanisms across distributed neural architectures.

12.1 Transition to Volume III

Volume I establishes the memory layer of the Trilogy. Volume II examines how compressed latent states may acquire symbolic stability. Volume III will then analyze how memory and symbolic structures may be coordinated through hierarchical routing.

Volume III: Routing Fields

Hierarchical Topic-Conditioned Routing for Mixture-of-Expert Language Models

Abstract

This article introduces *Hierarchical Topic-Conditioned Routing* (HTCR), an architectural framework for examining how routing mechanisms may improve coherence, cache locality, and computational organization within sparse Mixture-of-Experts (MoE) language models. Standard MoE architectures frequently rely on token-level routing decisions, which can fragment sequence-level structure and increase volatility in expert assignment during long-context inference. This token-local routing pattern may contribute to expert jitter, reduced cache reuse, and increased communication overhead in distributed execution environments.

HTCR proposes a two-tier routing mechanism. First, a macroscopic document- or segment-level router assigns local context segments of N tokens to a restricted pool of m candidate experts. Second, a microscopic token-level router dispatches individual tokens exclusively within that pre-allocated expert shortlist. This hierarchy is intended to preserve local semantic coherence while reducing unnecessary expert dispersion during long-sequence processing.

Building upon the memory substrates of *Volume I: Memory Fields* and the symbolic abstractions of *Volume II: Symbol Fields*, HTCR concludes the Trilogy by defining a dynamic macroscale routing infrastructure for distributed latent computation. This work provides a theoretical reference design, including mathematical mechanics for two-stage dispatch, structural load-balancing metrics, training stability regularizers, and empirical validation protocols for routing-aware neural architectures.

Keywords: Hierarchical Topic-Conditioned Routing (HTCR); Mixture-of-Experts (MoE); Cache Locality; Distributed Inference; Expert Jitter; Token Dispatch; Structural Regularization.

Chapter 13

Routing Fields and Hierarchical Topic-Conditioned Routing

13.1 Introduction

Volume III of *The Latent Cognition Trilogy* examines the architectural coordination layer of latent cognition. Volume I developed memory as a field of retrievable latent representations. Volume II examined how compressed latent states may acquire symbolic stability under curriculum and bottleneck pressure. Volume III addresses the next structural question: how can memory fields and symbol fields be coordinated across large-scale neural architectures whose computation is distributed across specialized components?

The central construct introduced here is *Hierarchical Topic-Conditioned Routing* (HTCR). HTCR proposes that routing should not be treated only as a token-local dispatch mechanism or a hardware efficiency optimization. Instead, routing may be understood as a structural principle through which computation is allocated, constrained, and stabilized across expert modules. In this framing, routing becomes the architectural mechanism that determines where latent information is processed, which expert pathways are activated, and how coherence is preserved across long sequences.

Standard sparse Mixture-of-Experts (MoE) systems often route tokens independently. This design allows conditional computation at scale, but it can fragment sequence-level structure, increase expert assignment volatility, and create communication patterns that are difficult to optimize in

distributed inference. HTCR introduces a two-stage routing process: first, a segment-level router identifies a restricted expert pool for a local context region; second, a token-level router dispatches individual tokens only within that constrained pool.

The purpose of this volume is not to claim that HTCR has been empirically validated. Rather, it develops a theoretical and systems-oriented framework for evaluating whether hierarchical routing can reduce expert jitter, improve cache locality, stabilize load balancing, and preserve semantic coherence across long-context computation.

13.2 Problem Space: Token-Level Routing and Architectural Fragmentation

Sparse MoE models increase model capacity by activating only a subset of expert parameters for each token. This provides an attractive scaling strategy: the total parameter count may grow substantially while the active computation per token remains bounded. However, token-level routing also introduces a coordination problem. If each token independently selects experts, adjacent tokens in a semantically coherent passage may be dispatched to unstable or widely dispersed expert sets.

This token-local routing pattern can produce several difficulties. First, expert assignment may fluctuate rapidly across neighboring tokens, creating what this volume refers to as *expert jitter*. Second, distributed execution may require expensive all-to-all communication when tokens in the same sequence are routed to experts located across different hardware partitions. Third, cache reuse may be weakened when expert activation patterns change too frequently across local context windows.

HTCR addresses these issues by adding a macroscopic routing stage before token-level dispatch. Instead of allowing every token to select from the full expert pool independently, the segment-level router selects a bounded shortlist of candidate experts for a local context segment. Token-level routing then proceeds within that shortlist. The hypothesis is that this hierarchical constraint may preserve semantic locality while reducing unnecessary routing dispersion.

Within the broader Trilogy, this problem has cognitive significance. Memory and symbolic abstraction are not sufficient unless the architecture can coordinate where and how they are processed. Routing therefore becomes the mechanism through which latent cognition is operationally organized across distributed computation.

13.3 Contributions

This volume contributes the following elements:

1. a theoretical framework for interpreting routing as architectural coordination rather than only conditional computation;
2. a two-stage HTCR mechanism combining macroscopic segment routing with microscopic token dispatch;
3. formal metrics for expert jitter, shortlist stability, routing entropy, load balancing, and communication cost;
4. a cache-locality analysis for long-context MoE inference;
5. an empirical evaluation protocol for comparing HTCR against flat token-routed MoE baselines;
6. a failure-mode taxonomy for hierarchical routing architectures; and
7. a synthesis of routing as the third layer of the Trilogy's memory–symbol–routing sequence.

13.4 Overview of the Volume

The remainder of this volume proceeds as follows. The next chapter situates HTCR within related work on sparse MoE architectures, token routing, load balancing, expert choice mechanisms, and long-context inference. Subsequent chapters define the propositions and falsification conditions, formalize the dual-stage routing mechanism, introduce stability and locality metrics, specify structural regularization terms, define expected failure modes, and outline empirical validation protocols.

Position within the Research Program

This manuscript constitutes **Volume III** of *The Latent Cognition Trilogy*. It develops the routing and architectural coordination layer of the broader research program by formalizing *Hierarchical Topic-Conditioned Routing* (HTCR) as a candidate mechanism for organizing distributed computation across specialized model components.
Volume I, *Memory Fields*, established the memory layer of the Trilogy by treating retrieval as a native operation over latent representational

structure. **Volume II**, *Symbol Fields*, examined the conditions under which compressed latent representations may acquire symbolic stability. Volume III extends this sequence by asking how memory and symbolic structures may be coordinated across large-scale architectures through hierarchical routing mechanisms.

The architectural foundation for the broader program remains *Differentiable Latent Indexing in LLMs: Toward Native Retrieval as a Cognitive Function*. That work supplies the retrieval-native substrate from which the Trilogy begins. The present volume completes the main sequence by shifting attention from memory and symbolic abstraction to the coordination of computation across expert modules, routing pathways, and distributed inference structures.

Accordingly, **Volume III** should not be read as a companion hypothesis paper. It is the third movement of the Trilogy. The companion hypothesis papers occupy a separate role: they isolate supporting claims without replacing the main sequence.

Chapter 14

Related Work

14.1 Sparse Mixture-of-Experts Models

Sparse Mixture-of-Experts architectures provide the immediate technical context for HTCR. Sparsely-Gated MoE introduced the use of trainable gating networks to route inputs to a sparse subset of expert feed-forward networks, enabling large increases in model capacity without activating all parameters for every input [22]. GShard extended this direction by combining conditional computation with automatic sharding, making large-scale sparse expert models more practical in distributed settings [16]. HTCR builds on this lineage but shifts the focus from capacity scaling to routing coherence. The central question is not simply whether sparse experts can increase model size efficiently, but whether routing can preserve sequence-level structure and reduce expert volatility during long-context processing.

14.2 Routing Simplification and Load Balancing

Switch Transformer simplified MoE routing by selecting a single expert per token and introduced techniques for improving stability and reducing communication overhead [5]. ST-MoE further examined stability and transferability in sparse expert models, emphasizing the practical difficulty of training sparse systems that remain reliable across tasks [29].
These works motivate HTCR's concern with balancing. A routing mechanism must avoid expert starvation, token dropping, and pathological concentration of traffic into a small number of experts. HTCR therefore includes macro-level entropy regularization and load-balancing penalties,

but adds a further constraint: balancing should occur without destroying local semantic coherence.

14.3 Large-Scale Sparse Language Models

GLaM demonstrated that sparsely activated MoE language models can scale model capacity while reducing training and inference cost relative to dense alternatives [4]. Mixtral later provided a modern sparse MoE language model in which each token is routed to a small subset of experts at each layer [11]. These systems show the practical relevance of sparse expert routing for language modeling at scale.
HTCR differs from these systems by introducing segment-conditioned expert shortlists. Rather than allowing every token to route against the full expert pool independently, HTCR constrains token-level routing through a local context-conditioned expert set.

14.4 Alternative Routing Mechanisms

Expert Choice Routing reverses the usual token-choice pattern by allowing experts to select tokens rather than requiring each token to select a fixed number of experts [28]. This approach directly addresses load imbalance and expert underutilization. HTCR is compatible with this broader design space because it also challenges unconstrained token-local routing, but it does so through hierarchical context conditioning rather than expert-side token selection.
The relevance of Expert Choice Routing for HTCR is conceptual as much as technical: routing policy is not a fixed implementation detail. It is an architectural design axis that can alter training dynamics, load distribution, specialization, and inference behavior.

14.5 Distributed MoE Training and Hardware Efficiency

Efficient sparse training requires alignment between routing dynamics and hardware execution. MegaBlocks reformulates MoE computation using block-sparse operations to avoid token dropping and reduce the mismatch between dynamic routing and GPU efficiency [7]. This line of work shows

that routing is not only a modeling question; it is also a systems problem involving memory layout, communication, batching, and kernel efficiency. HTCR extends this systems concern to long-context inference. If semantically related tokens can be routed through a more stable local expert pool, the resulting execution pattern may improve cache locality and reduce unnecessary all-to-all expert movement.

14.6 Long-Context Inference and KV-Cache Locality

Long-context inference introduces significant memory-management pressure because key–value cache size grows with sequence length. PagedAttention and vLLM address this problem by managing KV cache through paged memory structures, improving serving throughput and reducing memory waste [14]. Although HTCR does not replace cache-management systems, it targets an adjacent problem: the instability of expert activation patterns that can reduce locality in sparse expert inference.
The connection is therefore structural. KV-cache systems manage memory efficiently once computation is scheduled. HTCR attempts to make the computation schedule itself more locality-preserving by reducing expert dispersion within semantically coherent segments.

Chapter 15

Propositions and Falsification Conditions

The validity of Hierarchical Topic-Conditioned Routing as a theoretical routing mechanism rests on verifiable behavioral differences relative to flat token-routed MoE baselines (such as standard Mixtral or GShard configurations).

Proposition 1: Topological Cache Locality and Throughput

By constraining individual token execution trajectories to a segment-bounded expert shortlist, the HTCR framework will demonstrate a significant reduction in inter-device KV cache communication volume and yield higher token-processing throughput during long-context sequence execution.

Falsification Condition: If an HTCR model manifests equivalent or higher inter-node communication latency and lower generation throughput compared to an unconstrained token-MoE baseline when evaluated on continuous context lengths exceeding 32k tokens, the utility of segment-level clustering is falsified.

Proposition 2: Optimization Stability and Load Balancing

The combination of macro-level entropy regularizers and token-level gates will stabilize expert assignment trajectories, preventing severe expert starvation and balancing computational loads across distributed hardware partitions without sacrificing modeling capacity.

Falsification Condition: If tracking routing distributions during large-scale pre-training reveals persistent structural load imbalances, or if token-drop ratios match flat gating methods under maximum hardware throughput limits, the optimization balancing claims are ungrounded.

Chapter 16

The Dual-Stage Hierarchical Routing Architecture

Let a long-context sequence be partitioned into discrete segments S_j containing N sequential tokens, where the input token matrix is defined as $X \in \mathbb{R}^{N \times d}$.

16.0.1 Stage 1: Macroscopic Segment Routing

Before token-level evaluation begins, the local hidden representations from the previous layer are aggregated into a unified segment-level summary vector $s_j \in \mathbb{R}^d$ via strategic mean-pooling across the spatial dimension:

$$s_j = \frac{1}{N} \sum_{t=1}^{N} h_t^{(\ell-1)}. \tag{16.1}$$

This summary vector s_j is processed by a document-router $\mathcal{R}_{\text{doc}}$, parameterized by a two-layer Multi-Layer Perceptron (MLP), to compute an allocation profile over the entire pool of E total available experts:

$$G_{\text{doc}}(s_j) = \text{softmax}\left(W_2 \cdot \text{GeLU}\left(W_1 s_j + b_1\right) + b_2\right). \tag{16.2}$$

The top-m indices matching the highest activation values in $G_{\text{doc}}(s_j)$ are extracted to define the restricted active expert shortlist $\mathcal{M}_j = \{i_1, i_2, \ldots, i_m\}$ for the duration of segment S_j.

16.0.2 Stage 2: Microscopic Token Dispatch

For each individual token representation $h_t^{(\ell)}$ residing within segment S_j, the local token-router $\mathcal{R}_{\text{tok}}$ evaluates gating scores *exclusively* against the experts preserved in the shortlist $\mathcal{M}_j$:

$$G_{\text{tok}}\left(h_t^{(\ell)}\right) = \text{softmax}\left(\left\{w_k^\top h_t^{(\ell)} \;\middle|\; k \in \mathcal{M}_j\right\}\right). \tag{16.3}$$

Individual tokens are then dispatched to the top-k experts selected strictly from this local dictionary. Gradients flow end-to-end through both routing layers simultaneously via continuous backpropagation paths.

16.1 Routing Stability and Expert Locality Metrics

The central claim of HTCR is not merely that routing occurs in two stages. Its claim is that hierarchical routing should produce measurably more stable expert assignment patterns than unconstrained token-level routing. This requires explicit metrics for routing stability, expert dispersion, and local semantic coherence.

Let $\mathcal{M}_j$ denote the expert shortlist assigned to segment S_j. A simple shortlist stability score between adjacent segments can be defined as the Jaccard overlap:

$$\rho_j = \frac{|\mathcal{M}_j \cap \mathcal{M}_{j+1}|}{|\mathcal{M}_j \cup \mathcal{M}_{j+1}|}. \tag{16.4}$$

High values of ρ_j indicate that adjacent segments preserve similar expert pools, while low values indicate abrupt routing shifts. In long-context processing, moderate stability is desirable: the routing pattern should remain coherent across semantically continuous segments while still adapting when the topic or task structure changes.

Expert jitter can be defined as the complement of average shortlist stability:

$$J_{\text{expert}} = 1 - \frac{1}{J-1}\sum_{j=1}^{J-1} \rho_j. \tag{16.5}$$

A successful HTCR model should reduce J_{expert} relative to a flat token-routed MoE baseline, particularly on long documents with locally coherent semantic structure.

16.2 Routing Entropy and Load Distribution

Let p_e denote the empirical probability that expert e is selected by the macroscopic segment router across a batch or validation corpus. Macro-routing entropy can be defined as:

$$H_{\text{doc}} = -\sum_{e=1}^{E} p_e \log p_e. \tag{16.6}$$

Very low entropy indicates routing collapse, where the segment router repeatedly selects a small number of experts. Very high entropy may indicate insufficient specialization, where the router fails to form meaningful associations between segment structure and expert allocation. HTCR therefore seeks a controlled intermediate regime: enough diversity to prevent starvation, but enough structure to support specialization and locality.
Load deviation from a uniform target can be measured as:

$$\Delta_{\text{load}} = D_{KL}\left(\bar{G}_{\text{doc}} \,\|\, \mathcal{U}\right), \tag{16.7}$$

where $\bar{G}_{\text{doc}}$ is the average macro-routing distribution and $\mathcal{U}$ is a uniform distribution over the expert pool. This term also appears in the regularization objective and functions as a diagnostic measure during evaluation.

16.3 Communication Cost Model

In distributed MoE inference, routing decisions induce communication costs when tokens are dispatched to experts located on different devices. Let $c(e_a, e_b)$ denote the communication cost associated with moving activation or cache-relevant information between device partitions associated with experts e_a and e_b. For a flat token-routed MoE baseline, the communication cost over a segment may be approximated as:

$$\mathcal{C}_{\text{flat}}(S_j) = \sum_{t \in S_j} \sum_{e \in \mathcal{E}_t} c(d_t, d_e), \tag{16.8}$$

where $\mathcal{E}_t$ denotes the expert set selected for token t, d_t denotes the current device location of the token representation, and d_e denotes the device hosting expert e.
Under HTCR, token routing is constrained to the segment shortlist $\mathcal{M}_j$, producing:

$$\mathcal{C}_{\mathrm{HTCR}}(S_j) = \sum_{t \in S_j} \sum_{e \in \mathcal{E}_t \cap \mathcal{M}_j} c(d_t, d_e). \tag{16.9}$$

The expected systems advantage of HTCR can be expressed as:

$$\Delta_{\mathrm{comm}} = \mathbb{E}_{S_j}\left[\mathcal{C}_{\mathrm{flat}}(S_j) - \mathcal{C}_{\mathrm{HTCR}}(S_j)\right]. \tag{16.10}$$

A positive Δ_{comm} indicates that hierarchical routing reduces communication pressure relative to unconstrained token-level dispatch.

16.4 Cache Locality Objective

HTCR can also be evaluated through a cache-locality objective. Let A_t denote the expert activation set for token t. For adjacent tokens within a segment, local expert continuity can be defined as:

$$L_{\mathrm{cache}}(S_j) = \frac{1}{N-1} \sum_{t=1}^{N-1} \frac{|A_t \cap A_{t+1}|}{|A_t \cup A_{t+1}|}. \tag{16.11}$$

Higher values of L_{cache} indicate that neighboring tokens reuse similar expert pathways, increasing the likelihood of improved cache behavior and reduced expert-switching overhead. This metric does not replace hardware-level cache profiling, but it provides a model-level proxy for routing locality.

Chapter 17

Algorithmic Specification

To operationalize HTCR, Algorithm 17 defines a single hierarchical routing step for one segment within one MoE layer.

Algorithm 2: Hierarchical Topic-Conditioned Routing Loop
Input: segment hidden states $H_j^{(\ell-1)} = \{h_t^{(\ell-1)}\}_{t=1}^{N}$, expert pool $\mathcal{E}$, document router $\mathcal{R}_{doc}$, token router $\mathcal{R}_{tok}$, shortlist size m, token top-k value.
Output: routed expert assignments for all tokens in segment S_j.

1. Compute segment summary:

$$s_j \leftarrow \frac{1}{N} \sum_{t=1}^{N} h_t^{(\ell-1)}.$$

2. Compute macro-routing distribution:

$$G_{\text{doc}}(s_j) \leftarrow \mathcal{R}_{doc}(s_j).$$

3. Select segment expert shortlist:

$$\mathcal{M}_j \leftarrow \text{TopM}(G_{\text{doc}}(s_j), m).$$

4. For each token $t \in S_j$, compute token routing scores only over $\mathcal{M}_j$.
5. Select top-k experts for each token from $\mathcal{M}_j$.
6. Dispatch token representations to selected experts.
7. Aggregate expert outputs and return routed segment representations.

This procedure differs from flat token-level MoE routing by constraining the local search space before token dispatch. The document router establishes a coarse semantic prior over expert selection, while the token router preserves fine-grained flexibility inside that prior.

Chapter 18

Structural Regularization and Starvation Prevention

To prevent the macroscopic document-router from collapsing into a biased state where a subset of experts absorbs all workloads (starvation), we enforce an auxiliary balanced regularization penalty $\mathcal{L}_{\text{bal}}$ during pre-training:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{entropy}} + \lambda_2 \mathcal{D}_{\text{KL}} \left(\bar{G}_{\text{doc}} \parallel \mathcal{U} \right), \tag{18.1}$$

where $\bar{G}_{\text{doc}}$ is the average macro-routing distribution across the training batch, $\mathcal{U}$ represents a uniform distribution over the expert pool E, and hyperparameters λ_1, λ_2 govern the structural smoothing constraints. The shortlist ceiling parameter m is dynamically annealed throughout training, starting wide to maximize early discovery and tightening progressively to optimize final execution speed.

18.1 Full HTCR Training Objective

The balancing penalty alone does not define the full HTCR training process. A complete conceptual objective must account for task performance, macro-router entropy, load balancing, cache locality, and excessive routing volatility. A general HTCR objective can be written as:

$$\mathcal{L}_{HTCR} = \mathcal{L}_{task} + \lambda_b \mathcal{L}_{balance} + \lambda_h \mathcal{L}_{entropy} + \lambda_j \mathcal{L}_{jitter} + \lambda_l \mathcal{L}_{locality}. \tag{18.2}$$

Here, $\mathcal{L}_{task}$ is the standard language modeling or downstream task objective. The term $\mathcal{L}_{balance}$ penalizes expert starvation or excessive load

concentration. The entropy term $\mathcal{L}_{entropy}$ regulates the macro-router's selection profile. The jitter penalty $\mathcal{L}_{jitter}$ discourages unnecessary expert-set volatility across adjacent segments, while $\mathcal{L}_{locality}$ rewards stable expert reuse within semantically coherent regions.

The objective should not force all neighboring segments to use the same experts. Such rigidity would prevent adaptation to topic shifts. Instead, the goal is controlled continuity: stable routing when the local semantic structure is stable, and flexible rerouting when the segment content changes.

18.2 Shortlist Annealing

The shortlist ceiling parameter m may be dynamically annealed during training. Early in training, a larger value of m allows the model to explore a broader region of the expert pool. As training progresses, m can be reduced to encourage sharper specialization and lower communication overhead.

This annealing schedule creates a trade-off between exploration and locality. If m remains too large, HTCR degenerates toward flat token routing. If m becomes too small too early, the model may overcommit to unstable expert assignments and prevent useful specialization from emerging. The annealing path must therefore be treated as a tunable systems parameter rather than a fixed architectural constant.

Chapter 19

Expected Failure Modes

HTCR is a theoretical routing mechanism. Its value depends on whether hierarchical routing improves coherence, locality, and load balance without reducing modeling capacity. Several failure modes must therefore be evaluated explicitly.

19.1 Topic Collapse

The macroscopic segment router may collapse into selecting the same small expert subset for many unrelated segments. This would reduce communication volatility but destroy specialization. Topic collapse can be detected through low macro-routing entropy, poor expert diversity, and degraded performance on heterogeneous documents.

19.2 Routing Inertia

The opposite failure mode is excessive routing inertia. In this case, the segment router continues using the same expert shortlist even after the semantic structure of the document changes. Routing inertia may improve cache locality while harming adaptation. This failure is especially likely in documents with abrupt topic shifts, code-switching, multi-domain reasoning, or mixed-format inputs.

19.3 Segment-Boundary Error

HTCR depends on segment partitioning. If segment boundaries cut across coherent semantic units, the macro-router may assign inappropriate expert

pools. Boundary errors may occur in long documents where topic transitions do not align with fixed token windows. Adaptive segmentation may therefore be required in future implementations.

19.4 Expert Starvation

Despite balancing penalties, some experts may remain undertrained if the segment router repeatedly excludes them from shortlists. Expert starvation can reduce the effective capacity of the model and may produce brittle specialization. Evaluation must track not only token-level load but also macro-level shortlist inclusion frequency.

19.5 Over-Specialized Experts

HTCR may encourage experts to specialize too narrowly around segment categories. Such over-specialization may improve local performance while weakening generalization. This failure can be detected when expert-specific performance becomes highly domain-bound and transfer across related tasks declines.

19.6 Communication Overhead from Macro-Routing

The additional document-router stage may introduce overhead that outweighs the savings from reduced token-level dispersion. This would falsify the systems-efficiency motivation for HTCR. A fair evaluation must therefore include end-to-end latency, throughput, memory movement, and scheduling overhead, not only theoretical communication counts.

19.7 Reduced Flexibility under Mixed-Topic Contexts

A restricted expert shortlist may be inappropriate for segments containing multiple simultaneous reasoning modes. For example, a single passage may combine code, mathematical reasoning, factual recall, and dialogue. If m is too small, the shortlist may exclude experts needed for secondary but important operations. This failure motivates adaptive shortlist sizing.

Chapter 20

Empirical Evaluation Protocol

To validate the scalability and efficiency claims of the HTCR architecture, tracking scripts must execute four foundational forensic operations against equivalent baseline setups.

20.0.1 Distributed Throughput and Communication Profiles

The system must be evaluated on a distributed compute cluster across multiple GPU nodes. Scaling metrics must explicitly monitor the cross-device data volume transferred via standard `All-to-All` communication primitives. HTCR must demonstrate a significant reduction in communication overhead and near-linear scaling of text-generation throughput as context window lengths increase.

20.0.2 Annealing Path and Capacity Ablations

Evaluators must isolate the performance changes triggered by the schedule of the shortlist constraint parameter m. Systematic comparative training runs must trace the delta between a static, unannealed baseline and a dynamic scheduling path to map out the explicit convergence characteristics of the dual routers.

20.0.3 Expert Activation Entropy and Balancing Verification

The long-term distribution profiles of the expert array must be audited using standard structural allocation tracking. The load distribution profile must approximate a uniform target, demonstrating that the balancing penalty effectively prevents hardware starvation without degrading perplexity metrics on standard benchmarks such as MMLU and GSM8K.

20.0.4 Cache Locality and Memory Footprint Mapping

Finally, hardware trackers must register real-time L2/L3 cache hit-rates and high-bandwidth memory usage metrics during processing blocks. The HTCR architecture must achieve significant cache-retention benefits, demonstrating that document-level conditioning prevents standard expert cache-eviction loops.

Chapter 21

Discussion

21.1 Routing as Architectural Coordination

HTCR reframes routing as a mechanism of architectural coordination. In sparse expert systems, routing is often discussed in terms of efficiency: which experts should be activated, how loads should be balanced, and how computation can be distributed. These concerns remain central, but the Trilogy adds a broader interpretation. Routing determines how latent memory, symbolic abstraction, and specialized computation are organized across the model.

From this perspective, routing is not merely a dispatcher. It is a structural interface between representation and computation. A routing mechanism determines which parts of the architecture are allowed to participate in processing a given context, which pathways remain inactive, and how continuity is preserved across long sequences.

21.2 Relation to Volume I and Volume II

Volume I introduced memory fields as structured latent spaces accessible through differentiable retrieval. Volume II introduced symbol fields as compressed latent structures that may stabilize into reusable computational primitives. Volume III completes the sequence by examining how these structures may be coordinated across expert modules and distributed pathways.

The relation among the three volumes is therefore layered. Memory supplies retrievable structure. Symbolic stabilization supplies reusable abstraction. Routing supplies allocation and coordination. None of these

mechanisms is sufficient alone. A system with memory but no routing may retrieve relevant information without coordinating computation effectively. A system with symbolic abstractions but no routing may lack a mechanism for deploying those abstractions across specialized components. HTCR addresses this third layer.

21.3 Systems Interpretation

HTCR also has a practical systems interpretation. Sparse MoE models are not only mathematical architectures; they are executed on distributed hardware. Expert assignment determines communication patterns, memory access, cache behavior, and scheduling pressure. A routing policy that appears adequate at the level of model quality may become inefficient or unstable at the level of deployment.

The two-stage design of HTCR attempts to align semantic locality with systems locality. If a local context segment is semantically coherent, then constraining its expert pool may reduce unnecessary dispatch dispersion. This does not guarantee improved performance, but it defines a measurable hypothesis connecting model structure to hardware behavior.

21.4 Interpretability and Accountability

Routing decisions can become a form of interpretability evidence. If certain expert pools are repeatedly selected for identifiable segment types, the macro-router may reveal a coarse map of architectural specialization. However, this evidence must be interpreted carefully. Expert selection does not automatically imply semantic understanding, and routing patterns may reflect dataset artifacts, optimization biases, or hardware constraints.

For this reason, HTCR requires multiple diagnostic layers: expert utilization profiles, routing entropy, shortlist stability, cache-locality metrics, ablation studies, and human inspection of segment-to-expert associations. Routing accountability depends on correlating these diagnostics with task performance and failure behavior.

21.5 Limits of the Framework

HTCR remains a theoretical reference design. It does not prove that hierarchical routing will outperform flat token-level routing in all settings. Its

expected advantages are conditional: long contexts, semantically coherent segments, distributed expert placement, and meaningful expert specialization. In short contexts or highly mixed inputs, the hierarchy may provide little benefit or even reduce flexibility.

The framework should therefore be evaluated as a structured hypothesis. Its value lies in making routing coherence, expert jitter, and cache locality measurable rather than implicit.

Chapter 22

Conclusion

Volume III has developed *Hierarchical Topic-Conditioned Routing* as the routing and architectural coordination layer of *The Latent Cognition Trilogy*. Its central claim is that routing should be analyzed not only as an efficiency mechanism, but as a structural principle for organizing computation across specialized neural components.

HTCR proposes a two-stage routing process in which a macroscopic segment router selects a constrained expert shortlist and a microscopic token router performs local dispatch within that shortlist. This design is intended to reduce expert jitter, improve cache locality, stabilize load distribution, and preserve semantic coherence across long-context processing.

The framework defines falsifiable propositions, formal routing metrics, a communication-cost model, a cache-locality proxy, training regularizers, expected failure modes, and empirical evaluation requirements. It does not claim empirical validation. Rather, it specifies what such validation would require.

Within the Trilogy, Volume III completes the analytical progression from memory to symbol to routing:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

Volume I established retrieval-native memory as the first layer of latent cognition. Volume II examined symbolic stabilization under compression and curriculum pressure. Volume III has argued that these structures require architectural coordination through routing if they are to operate coherently at scale.

The broader implication is that cognition-like behavior in artificial systems should not be attributed to scale alone. It may depend on how memory

is structured, how abstractions stabilize, and how computation is routed across the architecture.

Conclusion: Toward a Layered Theory of Latent Cognition

The Latent Cognition Trilogy has developed a layered account of internal computation in contemporary neural architectures. Its purpose has not been to declare a completed theory of artificial cognition, nor to collapse memory, symbolic abstraction, and routing into a single explanatory mechanism. Rather, the Trilogy has proposed a structured pathway for analyzing how these mechanisms may interact within large-scale neural systems.

Volume I, *Memory Fields*, examined retrieval as a native dimension of latent computation. It argued that memory should not be treated only as an external retrieval layer or auxiliary document store, but may be modeled as a differentiable field of latent representations participating directly in the model's computational process.

Volume II, *Symbol Fields*, shifted the focus from retrieval to abstraction. It examined the conditions under which compressed latent representations may acquire symbolic stability under curriculum pressure, representational bottlenecks, and reuse constraints. In this framing, symbolic structure does not need to be imposed from outside the model as a classical symbolic system. It may emerge as a stabilized condition of compressed latent computation.

Volume III, *Routing Fields*, addressed the problem of architectural coordination. As neural systems scale across expert modules, memory structures, symbolic abstractions, and distributed inference environments, routing becomes more than an efficiency mechanism. It becomes a structural principle through which computation is allocated, constrained, and coordinated across the architecture.

Taken together, the three volumes describe a progression from memory to symbol to routing:

$$\mathcal{M} \implies \mathcal{S} \implies \mathcal{R}$$

This progression should not be interpreted as a strict causal sequence. Memory, symbolic abstraction, and routing may interact recursively and concurrently within implemented systems. The sequence adopted here is analytical: it provides a way to isolate each layer before examining their interdependence.

The broader research program remains open. The DLI-LLM architectural foundation provides the systems-level substrate for retrieval-native cognition. The Trilogy extends that substrate into a wider conceptual sequence. The companion hypothesis papers preserve narrower claims that support, constrain, or extend the main argument without overloading the central manuscript.

The central contribution of the Trilogy is therefore not a single architecture, benchmark, or empirical claim. Its contribution is a research grammar: a structured vocabulary for describing latent memory, symbolic stabilization, and routing coordination as interdependent dimensions of artificial cognition.

Future work should move in three directions. First, the proposed mechanisms require empirical implementation and controlled benchmarking against baseline Transformer, retrieval-augmented, memory-augmented, and MoE architectures. Second, the interpretability of latent memory, invented symbols, and routing decisions must be studied through probing, ablation, and diagnostic visualization. Third, the safety implications of internalized memory and opaque routing must be examined carefully, particularly in systems where retrieval, abstraction, and decision pathways become less externally inspectable.

The Trilogy should therefore be read as a foundation for further inquiry rather than a closed doctrine. Its argument is that cognition-like behavior in artificial systems may not emerge from scale alone, nor from isolated architectural tricks, but from the coordinated organization of memory, abstraction, and computation within latent space.

Bibliography

[1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*, 2015.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.

[3] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. *Advances in Neural Information Processing Systems*, 35:11079–11091, 2022.

[4] Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V. Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, pages 5547–5569, 2022.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Jakob Foerster, Ioannis Alexandros Assael, Nando de Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[7] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. In

Proceedings of Machine Learning and Systems, volume 5, pages 288–304, 2023.

[8] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing Machines. *arXiv preprint arXiv:1410.5401*, 2014.

[9] Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, et al. Hybrid computing using a neural network with dyanmic external memory. *Nature*, 538(7626):471–476, 2016.

[10] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[11] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

[12] Jeff Johnson, Matthijs racing Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[13] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Zettlemoyer Luke, and Kevin Clark. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[15] Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent

networks. In *Proceedings of the 35th International Conference on Machine Learning*, pages 2873–2882, 2018.

[16] Dmitry Lepikhin, Hyoukand Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Lewis, Sebastian Riedel, and Tim Rocktäschel. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[18] Tsendsuren Munkhdalai and Hong Yu. Neural semantic encoders. *Association for Computational Linguistics (ACL)*, pages 397–407, 2017.

[19] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Pan, Hua Wu, and Haifeng Wang. RocketQA: An optimized training approach to dense passage retrieval for open-domain question answering. *Conference of the National Chapter of the Association for Computational Linguistics (NAACL)*, 2021.

[20] Anna Rogers, Olga Kovaleva, and Anna Rumshisky. A primer in BERTology: What we know about how BERT works. *Transactions of the Association for Computational Linguistics*, 8:842–866, 2020.

[21] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. *International Conference on Machine Learning (ICML)*, pages 1842–1850, 2016.

[22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[23] Ron Sun. Desiderata for cognitive architectures. *Philosophical Psychology*, 17(3):327–353, 2004.

[24] Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000.

[25] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.

[27] Yuhuai Wu, Markus Rahnama, Angela Fan, Sainbayar Sukhbaatar, Jason Weston, and Armand Joulin. Memorizing transformers. *International Conference on Learning Representations (ICLR)*, 2022.

[28] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M. Dai, Zhifeng Chen, Quoc V. Le, and James Laudon. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems*, volume 35, pages 7103–7114, 2022.

[29] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.